

Getz CS-499 Milestone One Narrative

The artifact I am working on for the software design and engineering enhancement is a Company Data Dashboard for Grazioso Salvare, a search and rescue dog training organization. It allows staff to easily filter through data from the Austin animal shelter to identify dogs that match certain criteria for their programs. The dashboard originally used Python Dash and PyMongo. It now uses Dash for the front end and Flask for the backend, communicating via HTTP requests. It was created in CS-340 Client/Server Development and was first completed on 2/17/26.

I decided to include this artifact in my ePortfolio for a few reasons. It was one of the first projects that taught me how real applications are built. It taught me how databases are built and how applications can use them. While the original dashboard worked, it had a number of flaws such as hard-coded secrets and a monolithic architecture. This project gives me a unique opportunity to highlight a number of skills in software design and engineering. Improving this project shows I know how to take data from a database, use APIs, sort data, and present it in a way that is intuitive for a non-technical user to use. It shows I know how to improve already existing code. Not only that, but it highlights my ability to document software, which was a key part of the course. The artifact improvements will make the application more robust, secure, and easier for the user to interact with.

I believe that I did meet the course outcomes that I planned with this enhancement. The work I did turned out to be better than I thought it would have. I do not have any updates to my outcome coverage plans at this time. I showed that I have a security mindset by implementing a login prompt with Dash and using the API key to secure the APIs. I showed that I can implement computer solutions that deliver value and accomplish a goal by taking a holistic look at what the

dashboard does. Grazioso Salvare staff do not need to call the Austin animal shelter or filter through new animals manually. They can simply load the dashboard and load a filter instantly. The various documentation and code comments I created highlight my ability to build collaborative environments. If another developer ever wanted to add onto the dashboard, they would have an easier time. Because I have not fully implemented the database yet, there are still some changes I will need to make after I complete the database category. For example, the Dash Leaflet map does not show the animals' information because I need to set up the schema for the data when I get it from the real Austin animal shelter API.

The process of enhancing this artifact was very enjoyable. I faced many challenges and learned a lot from the process. There were many things that I did not know how to do, so I had to do some research. Before working on this project, I had never used Flask; however, because I am fluent in Python, it was very easy to pick up after looking at examples. I also had never worked with environment variables before and had to do some research on how to use them correctly. I underestimated how much work the enhancement would be. I assumed that I could keep the CRUD and Dash file the same, but this did not turn out to be the case. I had to make many changes to these files. To test my application, I set up a simple MongoDB on Atlas and had to learn how to connect to it and query it correctly. This was the first time I had done this, and setting up the database was quite easy, but getting it to work end-to-end was much harder. I also made a few small changes to the UI and UX, such as adding a loading icon for the data table, adding an alt tag to the logo, and creating better error handling, so the dashboard doesn't crash on the user.

I learned that adding authentication with Dash is very easy. On top of adding security for my API endpoints, I added a login popup to access the dashboard, adding a small layer of

security. My original plan was to create a full login page because I thought I needed it. Having this popup to enter login information for an in-house dashboard is more than enough. I also reevaluated using JWTs to secure my APIs. Because all users for this application will have the same permissions, JWTs are not necessarily needed. Instead, I used an API key to secure the endpoints. Before any request is made, the API key will be requested from the front end. If it is not found, then it will send a 401 unauthorized access response. This is much simpler to implement and more than enough security for this application. However, it relies on an attacker not getting the API key, so environment variable management is important. This enhancement was my first time doing many things, and the experience has made me a much stronger developer.